

CITS3005 Homework 2 report

1. Terraria – overview and knowledge graph considerations

1.1. Overview

Terraria is a popular 2-dimensional sandbox game; a player spawns into a new world, filled with unique biomes, enemies, gear, weapons, and bosses.

Players can *craft* from resources gathered in their world, and from resources *dropped* from bosses/enemies; the goal is to craft exceedingly powerful gear to progress throughout the game; as the player gets stronger, they also take on stronger bosses.

The game is split into two major phases; *pre-hardmode*, and *hardmode*. The player spawns in a pre-hardmode world; when the player defeats the final boss in pre-hardmode, it forces the world to transition into hardmode, which is ultimately the more challenging portion of the game. New resources, bosses, and gear is introduced here.

As such, certain aspects of Terraria can be represented through a knowledge graph; although not all of its components will be represented in the graph, due to scalability and complexity concerns.

1.2. Knowledge graph implementation

Terraria has several relations which can be represented within a knowledge graph, importantly, a knowledge-graph is a set $G \subseteq V \times E \times V$, where V is the set of *entities*, and E the set of *relations*. Following this, we can represent the following relations:

- “*x dropped_from y*”; an entity x is dropped from an entity y .
 - E.g., “*Stinger dropped_from Hornet*”.
- “*x crafted_from y*”; an entity x is crafted from an entity y .
 - E.g., “*Zenith crafted_from Terrablade*”.
- “*x crafted_at y*”; an entity x is crafted at entity y .
 - E.g., “*Terrablade crafted_at HardmodeAnvil*”.
- “*x acquired_from y*”; entity x is acquirable from an entity y ; here, y would usually denote a location.
 - E.g., “*StarFury acquired_from Chest*”.
- “*x is_a y*”; shows that x is a version of y ; this is needed for representing an “either” model.
- “*x game_phase y*”, where entity x first appears in the game phase y .
- “*x health y*” relation assigning health literal to bosses.
- “*x damage y*”; relation assigning a damage literal to weapons.

Note: the health and damage relations exist purely to assign literal values to certain entities.

Furthermore, the following *types* can be represented in the knowledge graph (note: a type is a grouping of entities with similar properties/attributes):

- *Boss*; very strong enemies which drop powerful items upon defeat.
- *Enemy*; a standard monster; they are generally aggressive against the player and drop items when defeated.
- *Weapon*; used by the player to defeat enemies; they can be crafted from materials, dropped from enemies/bosses, or acquired from special regions.
- *Material*; an item used solely for crafting.
- *CraftingSource*; which is the crafting station required for specific items to be crafted at.
- *Location*; a specific region in the world.
- *Progression*; the current game-phase of the world (either pre-hardmode, or hardmode).

The *is_a* exists to represents an “*either*” model in the knowledge graph. For example, given the two evil swords *Lightsbane*, and *Blood Butcherer*, the crafting recipe for *Night’s Edge*, calls for only one of these swords, so:

“*NightsEdge crafted_from Lightsbane*”

“*NightsEdge crafted_from BloodButcherer*”

Would imply that both the *Lightsbane* and *Blood Butcherer* are required in the crafting tree for the *Night’s Edge*, which is not true; in the actual crafting tree, just one of these swords is required.

So, in this case, we instantiate “*AnyEvilSword, type, Weapon*” as a triple; this creates the entity that shall represent both. After, the *is_a* relation is used here, creating a relationship between the evil swords and *AnyEvilSword*:

“*Lightsbane is_a AnyEvilSword*”

“*BloodButcherer is_a AnyEvilSword*”

The same methodology is applied for the different entities of anvils, forges, altars, and mechanical bosses.

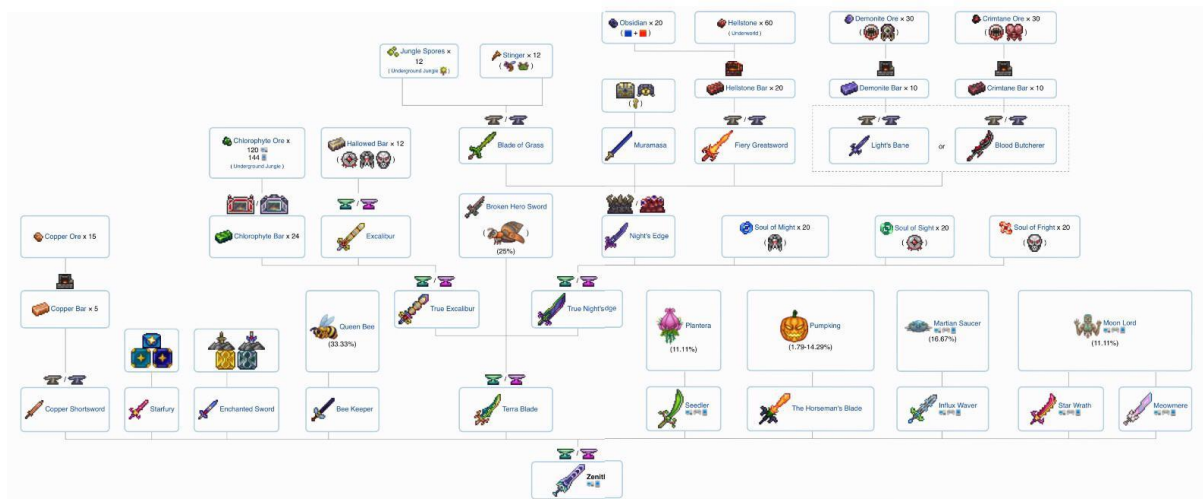
2. Knowledge graph queries – possible answers to common questions

Terraria often has extremely complex crafting trees, spanning over several weapons, and materials. A knowledge graph shall allow someone to:

- Look up the drops from a boss, and specifically, or find a specific boss that drops a specific item.
- Identify all crafting materials required to craft a specific item and even find *all* crafting materials for each component to craft the desired item, done recursively.
- Return all bosses, unlocked within a specific game phase.

A Wiki page may answer the above questions, but a knowledge graph is the ultimate representation of the relationships between specific entities, which are called *triples*; these triples can be dynamically queried in the knowledge graph. Additionally, in a knowledge graph is capable of deriving new facts from previously established facts. Wiki's are not capable of doing these.

Consider the Zenith crafting tree (which is provided below):



Observably, it is quite large.

The graph itself only stores bare facts; however, we are able to query the graph such that new facts can be derived by *chaining*; essentially, this knowledge graph allows us to execute a single query, it follows the relations recursively until it finds the required data; it binds to the specified variables in the query and returns.

By adding the “+” operator to the end of the relation in the query, it will return *every related item*, too.

Furthermore, *Aggregation* is also useful; it enables the execution of number-specific queries, such as:

- Yielding weapons whose damage exceeds a certain value.
- Returning bosses whose hp is greater than some specified amount.
- Returning the average hp across all bosses.